INCIDENT EVIDENCE WALKTHROUGH

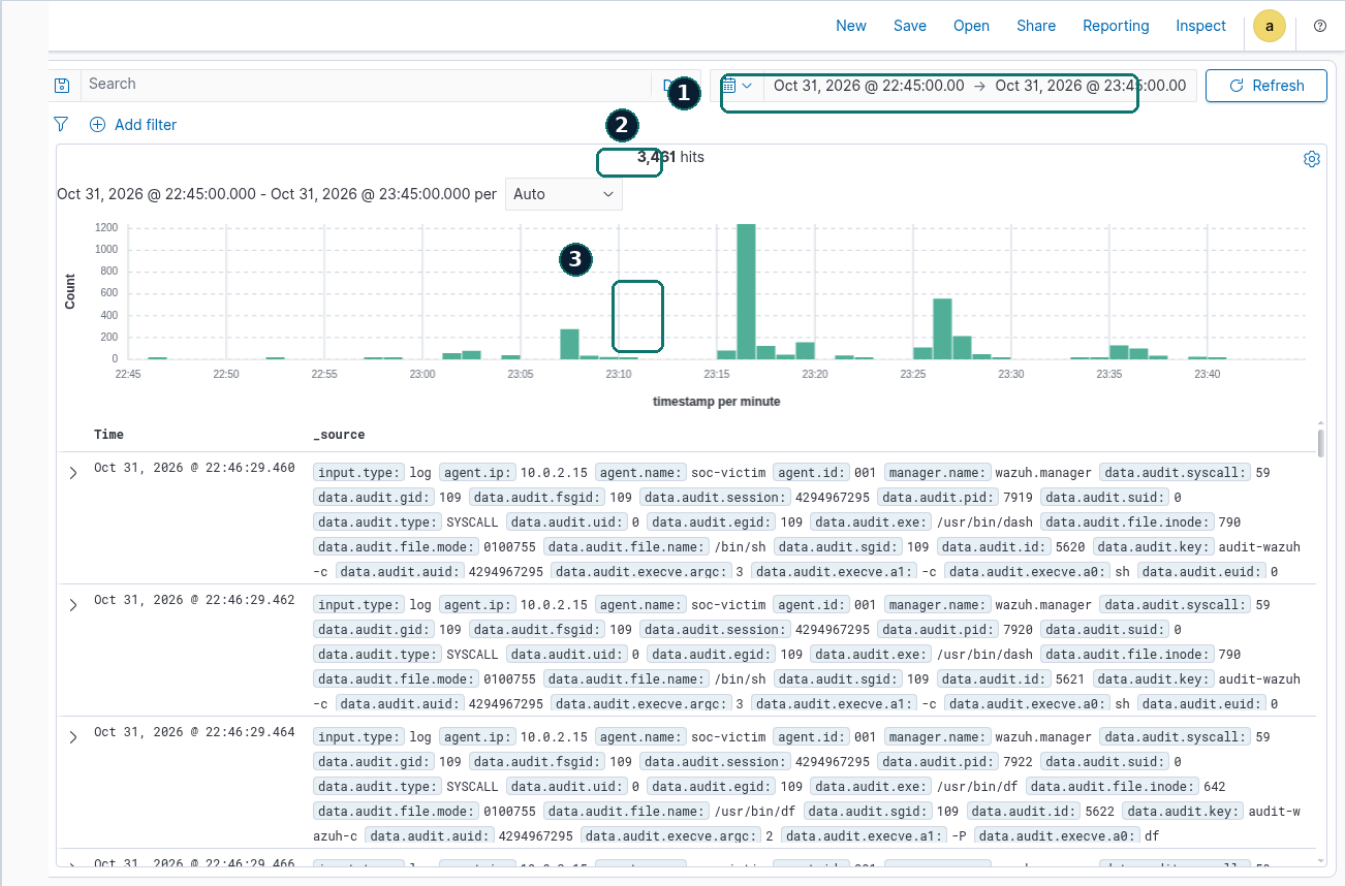
Cron and systemd persistence as root

INC-004 · Critical severity · 31 October 2026 · soc-victim

2	4	0	98 s
persistence paths	custom alerts	uid of proof files	to cleanup

1. Start with the hour

One hour around the persistence. The incident is a short cluster at 23:17, and the cleanup follows at 23:19.



1. Full hour in UTC, empty query.
2. The cluster at 23:17 is small compared with routine audit volume.
3. Deletions at 23:19 are the response, not the attack.

WHAT TO LOOK FOR IN THE LOGS

Two mechanisms

A cron file and a systemd unit appear within three seconds.

Root execution

Both run as root, which raises severity.

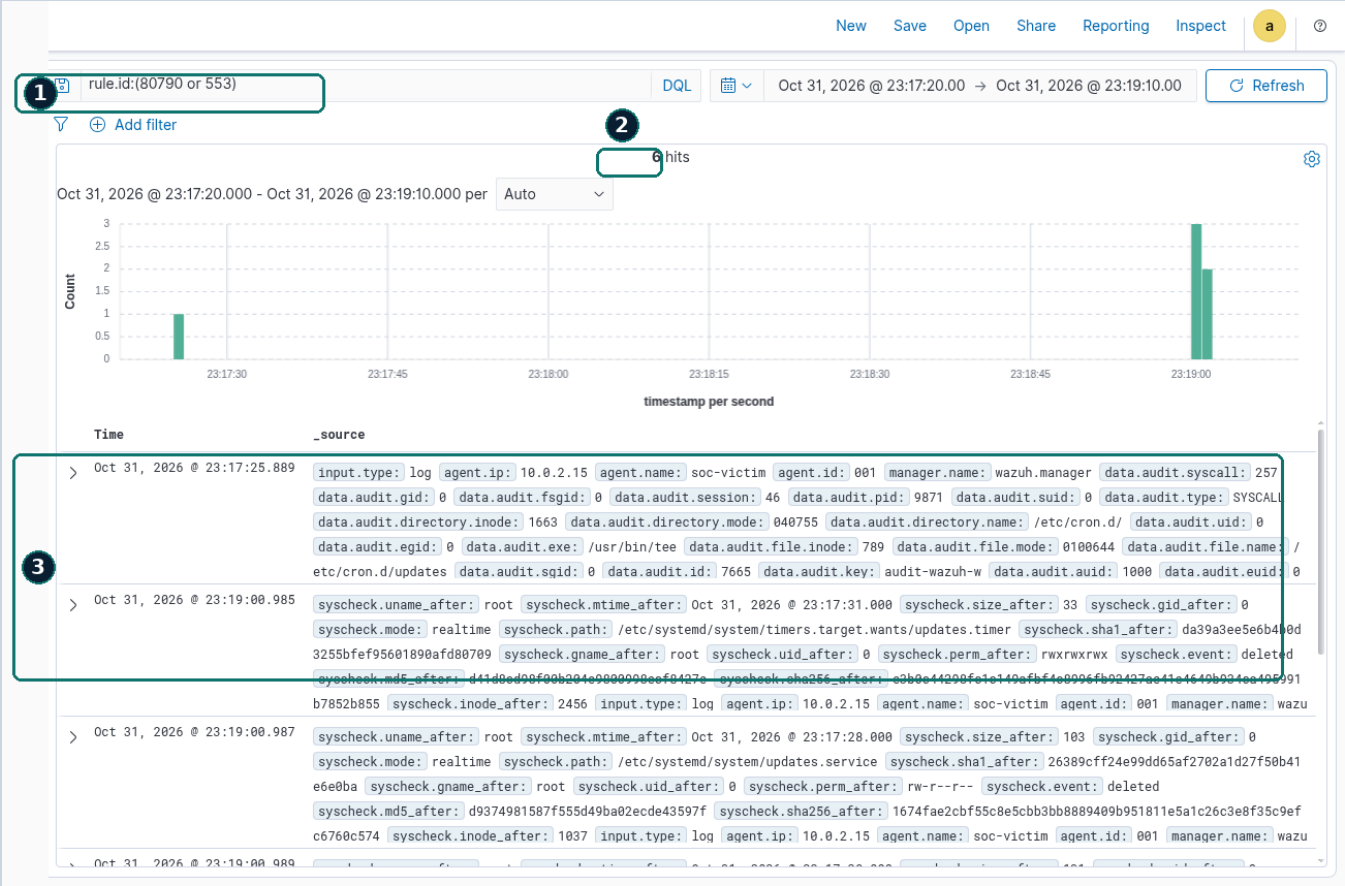
Cleanup

Deletions 90 seconds later are part of the ticket.

> A persistence alert is only half the story. The proof file shows whether it ran.

2. Stock rules see the files appear

Rule 80790 records the cron file creation, and 553 records the deletions during cleanup.



1. Query on the stock audit rule and the deletion rule.
2. The cron file is created at 23:17:25 and deleted at 23:19:00.
3. The rows name the path and the command.

WHAT TO LOOK FOR IN THE LOGS

Path

/etc/cron.d/updates is a root cron file.

Sequence

Create, then delete 95 seconds later.

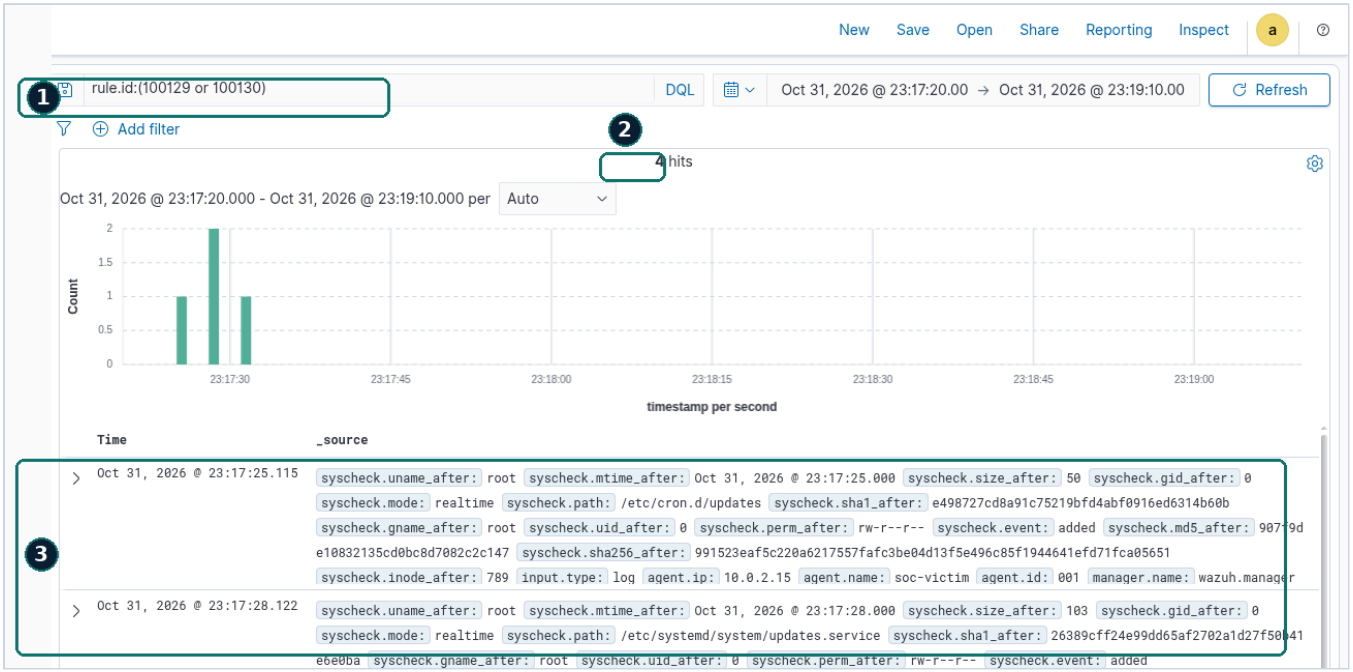
Actor

sudo from the analyst account, not the attacker.

> Deletion alerts are useful. They show the response and can hide anti-forensics if the actor is unknown.

3. Apply the custom rules

100129 covers cron files and 100130 covers systemd units and enable links. Together they name both persistence paths.



1. Query on both custom rules.
2. Four hits: one cron file and three systemd changes.
3. The systemd rows include the unit body and the enable link.

WHAT TO LOOK FOR IN THE LOGS

Cron

A file under /etc/cron.d is root-scheduled execution.

Systemd

Unit plus symlink. The symlink is what makes it run at boot.

Both paths

Two independent persistence mechanisms in three seconds.

> The systemd rule fires on enable links too, so it is noisier. Read the path before the count.

4. The persistence timeline

The exported chain shows creation, enable, execution, and cleanup.

soc-lab, INC-004, alert chain, UTC

```
$ jq -r '.[] | select(.rule.id=="80790" or .rule.id=="100129"
      or .rule.id=="100130" or .rule.id=="553")
      | "\(.timestamp[11:19])  \(.rule.id)  \(.rule.description)"' \
      docs/evidence/INC-004-alerts.json
23:17:25  100129  FIM: cron file added or modified (possible persistence)
23:17:25  80790  Audit: Created: /etc/cron.d/updates.
23:17:28  100130  FIM: systemd unit or enable-link changed (possible persistence)
23:17:28  100130  FIM: systemd unit or enable-link changed (possible persistence)
23:17:31  100130  FIM: systemd unit or enable-link changed (possible persistence)
23:19:00  553  File deleted.
23:19:00  553  File deleted.
23:19:00  553  File deleted.
23:19:01  553  File deleted.
23:19:01  553  File deleted.
```

1. 100129 at 23:17:25: the cron file.
2. 100130 at 23:17:28 and 23:17:31: unit body and enable link.
3. 553 at 23:19: deletions during cleanup.

ALERT MIX

RULE	ALERTS	WHAT IT ADDS
100129	1	Cron file added
100130	3	Unit body and enable links
80790	1	Audit record of the cron file
553	5	Deletions during cleanup

> Persistence that runs as root is Critical even when the proof file is trivial. The mechanism, not the payload, drives severity.

5. The persistence files

Three files define both mechanisms. Read them before deletion.

soc-lab, INC-004, persistence files

```
$ cat docs/evidence/INC-004-files.txt
# /etc/cron.d/updates
* * * * * root /usr/bin/id > /tmp/cron-proof 2>&1
# /etc/systemd/system/updates.service
[Unit]
Description=Updates
[Service]
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/systemd-proof 2>&1"
# /etc/systemd/system/updates.timer
[Unit]
Description=Updates timer
[Timer]
OnBootSec=30s
OnUnitActiveSec=60s
Unit=updates.service
[Install]
WantedBy=timers.target
```

1. The cron entry runs /usr/bin/id as root every minute.
2. The service is a oneshot that writes id output to /tmp.
3. The timer starts at boot and repeats every 60 seconds.

WHAT TO LOOK FOR IN THE LOGS

Minute cron

A root job that repeats every minute is not maintenance.

Oneshot root

ExecStart runs a shell command as root.

Install target

WantedBy=timers.target makes it persist across reboots.

> Capture file contents and hashes before removal. A path alone does not show intent.

6. Files, hashes, and proof

The proof files are the difference between persistence that was configured and persistence that ran.

soc-lab, INC-004, hashes and proof

```
$ cat docs/evidence/INC-004-hashes.txt docs/evidence/INC-004-proof.txt
991523eaf5c220a6217557fafc3be04d13f5e496c85f1944641efd71fca05651  /etc/cron.d/updates
1674fae2cbf55c8e5cbb3bb8889409b951811e5a1c26c3e8f35c9efc6760c574
/etc/systemd/system/updates.service
b58a06d878ddb229f8bfed5eaebf1d32341ee345d0897b765868f7d16c6f0129
/etc/systemd/system/updates.timer
cron:    uid=0(root) gid=0(root) groups=0(root)
systemd: uid=0(root) gid=0(root) groups=0(root)
```

1. Three SHA-256 values identify the artifacts.
2. Both proof files read uid=0(root), so both mechanisms executed.

WHAT TO LOOK FOR IN THE LOGS

Hashes

Record all three before removal.

Proof files

Read /tmp/cron-proof and /tmp/systemd-proof.

Root

uid=0 confirms the severity.

> If the proof files were empty or missing, the finding would be attempted persistence, not execution.

7. How to capture this evidence

1. **Start broad**
   One hour around the event.

2. **Stock rules**
   File creation and deletion.

```
rule.id:(80790 or 553)
```

3. **Custom rules**
   Cron and systemd.

```
rule.id:(100129 or 100130)
```

4. **Save file contents**
   Cron file, service, timer.

```
cat /etc/cron.d/updates /etc/systemd/system/updates.service \
  /etc/systemd/system/updates.timer
```

5. **Record hashes and proof**
   Hashing only reads the files. It does not run or disable the units.

```
sha256sum /etc/cron.d/updates /etc/systemd/system/updates.*
cat /tmp/cron-proof /tmp/systemd-proof
```

> Read the unit body and the enable link separately. A unit file that is not enabled may never run.

8. What to look for

Two mechanisms at once

Cron and systemd in the same minute is a deliberate persistence attempt.

Root execution

A root oneshot or cron job writes a proof file.

Enable link

The symlink under timers.target is what survives a reboot.

Cleanup timing

Deletion 90 seconds later suggests a controlled test or a careful actor.

Proof

uid=0 in the output makes it Critical.

COMMON PITFALLS

Systemd rule is noisy

Enable links from packages also fire. Check the path and the unit body.

Deletion is not always good

If the actor, not the admin, deleted the file, treat it as anti-forensics.

Unit without link

A unit file alone may never execute. Confirm the enable link.

> Verdict: Critical severity, contained. Both mechanisms ran as root and were removed, with hashes and proof files preserved.